

SmartFuzz

Security Assessment Report

Target: http://testfire.net/

Scan Date: 2026-05-18 06:17:55 UTC

Scan ID: ece7b4b6-a718-4a65-b732-257f61337902

1. Executive Summary

Target URL	http://testfire.net/
Scan Date	2026-05-18 06:17:55 UTC
Vuln Classes	SQLi, XSS, RCE, SSRF, Command Injection
Overall Risk	Critical

4	2	0	2	0	1	100
Total	Critical	High	Medium	Low	Forms	Payloads

2. Findings

Summary

#	Vulnerability	Parameter	Severity	Response Snippet
1	SSRF	content	Critical	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC " -//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/x...
2	SSRF	content	Critical	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC " -//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/x...
3	XSS	content	Medium	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC " -//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/x...
4	XSS	content	Medium	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC " -//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/x...

Detailed Findings

#1 SSRF — content		Critical
URL	http://testfire.net/index.jsp?content=http%3A%2F%2F169.254.169.254%2Flatest%2Fmeta-data%2F	
Status	200	
Resp Time	0.297s	
Timestamp	2026-05-18 06:17:36	
Payload	http://169.254.169.254/latest/meta-data/	
Evidence	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" " http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet	
Fix	1. Whitelist allowed destination domains/IPs and reject everything else. 2. Block requests to 169.254.0.0/16 (AWS metadata), 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. 3. Disable HTTP redirects, or re-validate the final destination URL after each redirect. 4. Use a dedicated egress proxy that enforces the allowlist. 5. Avoid accepting user-supplied URLs entirely where possible. 6. Reference: OWASP Top 10 A10:2021 — SSRF	

#2 SSRF — content

Critical

URL http://testfire.net/index.jsp?content=http%3A%2F%2F169.254.169.254%2Flatest%2Fuser-data%2F

Status 200

Resp Time 0.265s

Timestamp 2026-05-18 06:17:37

Payload http://169.254.169.254/latest/user-data/

Evidence <!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet

Fix 1. Whitelist allowed destination domains/IPs and reject everything else. 2. Block requests to 169.254.0.0/16 (AWS metadata), 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. 3. Disable HTTP redirects, or re-validate the final destination URL after each redirect. 4. Use a dedicated egress proxy that enforces the allowlist. 5. Avoid accepting user-supplied URLs entirely where possible. 6. Reference: OWASP Top 10 A10:2021 — SSRF

#3 XSS — content

Medium

URL http://testfire.net/index.jsp?content=%27%3Balert%28%27XSS%27%29%2F%2F

Status 200

Resp Time 0.266s

Timestamp 2026-05-18 06:17:19

Payload ' ;alert('XSS')//

Evidence <!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet

Fix 1. HTML-encode all user-controlled output before rendering (use your framework's auto-escaping). 2. Implement a strict Content-Security-Policy (CSP) response header. 3. Set HttpOnly and Secure flags on all session cookies. 4. Validate and sanitize input on the server side — treat all input as untrusted. 5. Use modern frameworks (React, Vue, Angular) that escape output by default. 6. Reference: OWASP Top 10 A03:2021 — Injection

#4 XSS — content		Medium
URL	http://testfire.net/index.jsp?content=%3Cdetails+open+ontoggle%3Dalert%28%27XSS%27%29%3E	
Status	200	
Resp Time	0.281s	
Timestamp	2026-05-18 06:17:19	
Payload	<details open ontoggle=alert('XSS')>	
Evidence	<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet	
Fix	1. HTML-encode all user-controlled output before rendering (use your framework's auto-escaping). 2. Implement a strict Content-Security-Policy (CSP) response header. 3. Set HttpOnly and Secure flags on all session cookies. 4. Validate and sanitize input on the server side — treat all input as untrusted. 5. Use modern frameworks (React, Vue, Angular) that escape output by default. 6. Reference: OWASP Top 10 A03:2021 — Injection	

3. Remediation Reference

SSRF

1. Whitelist allowed destination domains/IPs and reject everything else.
2. Block requests to 169.254.0.0/16 (AWS metadata), 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
3. Disable HTTP redirects, or re-validate the final destination URL after each redirect.
4. Use a dedicated egress proxy that enforces the allowlist.
5. Avoid accepting user-supplied URLs entirely where possible.
6. Reference: OWASP Top 10 A10:2021 — SSRF

XSS

1. HTML-encode all user-controlled output before rendering (use your framework's auto-escaping).
2. Implement a strict Content-Security-Policy (CSP) response header.
3. Set HttpOnly and Secure flags on all session cookies.
4. Validate and sanitize input on the server side — treat all input as untrusted.
5. Use modern frameworks (React, Vue, Angular) that escape output by default.
6. Reference: OWASP Top 10 A03:2021 — Injection

4. Disclaimer

This report was generated automatically by SmartFuzz. Results should be reviewed and validated by a qualified security professional before acting upon them. False positives may be present. This tool is intended for use only against systems you own or have explicit written authorisation to test. Unauthorised testing is illegal and unethical.

Generated by SmartFuzz · 2026-05-18 06:17:55 UTC